A Reproducible Software-Defined CAN Testbed for Automotive Cybersecurity and Forensic-Readiness Analysis

Osvaldo Janeri Filho
Independent Researcher / Digital Forensics Practitioner
Fortaleza, Ceará, Brazil
Email omitted for blind or camera-ready submission

***Abstract*—Controller Area Network (CAN) remains common in production and legacy vehicles, but it does not provide native authentication, confidentiality, or sender verification. This paper presents a reproducible, software-defined CAN testbed for automotive cybersecurity and forensic-readiness analysis. The artifact packages labeled internal experiments, a 10-run intensity matrix, external ICSim trace ingestion, didactic spoof-packet injection, wall-clock timing capture, python-can virtual-bus experiments, normalized datasets, source metadata, SHA-256 manifests, validation scripts, GitHub Actions, and observer-safe browser demos. Across internal labeled experiments, the testbed generated 477,738 CAN frames. External-source compatibility is demonstrated by importing 6,158 frames from the public ICSim `sample-can.log` trace and replaying the same trace through a python-can virtual bus. Additional timing packages record 1,167 wall-clock frames, 1,750 generated python-can virtual-bus frames, and 6,158 replayed ICSim frames, all with zero negative inter-arrival values. A rule-based triage heuristic is retained only as a label-consistency and forensic-screening sanity check, not as a real-world intrusion detection benchmark. The contribution is an evidence-preserving reproducibility workflow rather than a claim of physical CAN fidelity or operational IDS performance.**

***Index Terms*—automotive cybersecurity, CAN bus, forensic readiness, reproducible research, testbed, ICSim, virtual CAN**

I. Introduction

Modern vehicles use multiple electronic control units connected through in-vehicle networks. CAN was designed for robust and low-cost communication, but not for cryptographic authentication or strong origin verification. A receiver typically processes frames according to identifiers and payload semantics, not authenticated sender identity.

This creates a useful teaching and forensic-analysis problem: students and practitioners need to understand spoofing, flooding, replay, evidence preservation, and trace interpretation, but safe and reproducible access to automotive networks is uneven. Real vehicles can be expensive and risky to manipulate; used ECUs may require proprietary pinouts and immobilizer pairing; and commercial hardware-in-the-loop (HIL) platforms often exceed small-lab budgets.

This paper addresses that gap with a reproducible software-defined CAN testbed designed for automotive cybersecurity and forensic-readiness analysis. The goal is explicitly bounded: the artifact does not claim physical CAN fidelity, production-vehicle realism, or operational intrusion-detection performance. Instead, it provides a reproducible workflow for generating and preserving traces, importing external traffic, injecting didactic events, validating hashes, and replaying evidence safely in a browser.

The paper is organized around four research questions:

- **RQ1:** Can a low-cost software-defined CAN testbed preserve evidence needed for later review?
- **RQ2:** Can the workflow generate controlled spoofing and flooding variations with explicit labels and manifests?
- **RQ3:** Can the same pipeline ingest and preserve an external CAN trace without changing the evidence schema?
- **RQ4:** Can observers review the resulting artifacts safely without live CAN hardware or vehicle access?

The contributions are: (i) a reproducible software-defined CAN testbed architecture and evidence workflow; (ii) labeled internal experiments with baseline, spoofing, flooding, and recovery phases; (iii) a 10-run intensity matrix for parameter-sensitivity evidence; (iv) external-source compatibility through ICSim trace ingestion; (v) didactic spoof-packet injection for speed, door, and turn-signal visualization; and (vi) a public artifact with DOI, SHA-256 validation, normalized CSV files, a data dictionary, and observer-safe demos [1].

II. Background and Related Work

Classic automotive security studies showed that in-vehicle networks can be manipulated after attackers obtain access to relevant interfaces [2]–[4]. Subsequent work surveyed CAN security challenges such as replay, fabrication, bus availability, and proprietary signal interpretation [5], [6]. Standards including ISO/SAE 21434 and UNECE R155 provide broader cybersecurity engineering and management context [7], [8].

Educational and experimental tools include ICSim, which provides a visual CAN training environment, and Caring Caribou, which supports CAN and diagnostic exploration [9]–[11]. Physical and hybrid benches such as VitroBench and CAN bus security testbeds support stronger realism but require more hardware and setup effort [12], [13]. Public datasets such as ROAD, can-train-and-test, and can-sleuth support IDS research and dataset analysis [14]–[16].

The gap targeted here is not the absence of simulators or datasets. It is the lack of a compact workflow that combines

TABLE I
POSITIONING AGAINST RELATED ARTIFACTS

Artifact	SW	Data	Hash	Demo	Real.
ICSim	Yes	Sample	No	Visual	Low
Caring Caribou	Partial	No	No	No	Tooling
ROAD / can-train	N/A	Yes	No	No	Trace
VitroBench / HIL	No	Limited	Limited	No	Higher
Proposed	Yes	Yes	Yes	Yes	Future HIL

TABLE II
ARTIFACT REPRODUCIBILITY COMPARISON

Artifact	DOI	Hashes	Schema	Safe demo
ICSim	No	No	Sample	Yes
ROAD	Dataset	No	Yes	No
can-train-test	Dataset	No	Yes	No
Proposed	Yes	Yes	Yes	Yes

low-cost software-only experimentation, explicit forensic evidence packaging, source-trace metadata, hash validation, DOI-backed availability, and safe observer replay. Table I positions the artifact accordingly.

III. TESTBED AND EVIDENCE WORKFLOW

A. Architecture

The bus layer uses `python-can` virtual channels and can be migrated to SocketCAN/`vcan`. Simulated ECUs generate periodic engine/cluster, body, display, and background frames. Attack modules inject fabricated state or randomized high-cardinality traffic. External importers normalize upstream traces into the same schema. The logger and packaging scripts produce raw/source logs, normalized CSV files, summaries, and manifests.

B. Forensic-Readiness Definition

For this artifact, forensic readiness means that experiments are designed so relevant evidence is already organized for later review. Each package should preserve traceability, integrity, repeatability, normalization, and reviewability. Table III maps these properties to concrete files.

C. Ethical Scope and Safety Boundary

All experiments reported here are authorized and software-only. No vehicle, production ECU, CAN adapter, external network, or operational system is targeted. The demos replay stored artifacts and do not execute live attacks. The repository explicitly frames the work as an educational and forensic-readiness artifact; it should not be used against vehicles, ECUs, networks, or systems without explicit authorization.

IV. EXPERIMENTAL DESIGN

Table IV summarizes the evidence packages used in v6. Experiments 005 and 006 provide internal labeled traffic. Experiment 007 expands this into a 10-run matrix. Experiment 008 imports an external ICSim trace. Experiment 009 injects

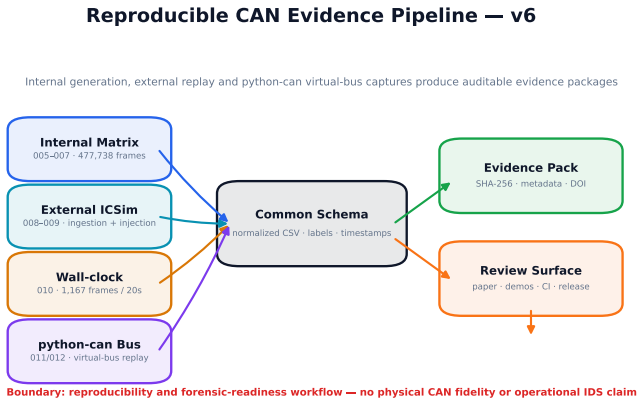

Fig. 1. Reproducible CAN evidence pipeline. Internal runs, external traces, and wall-clock capture feed a common normalization and evidence-packaging workflow with manifests, DOI-backed artifacts, and observer-safe demos.

TABLE III
FORENSIC-READINESS MAPPING

Requirement	Implemented artifact	Boundary
Traceability	Input plans, injection plans, source metadata	Scenario provenance
Integrity	SHA-256 manifests	File integrity
Repeatability	Scripts, seeds, matrix configs	Software behavior
Normalization	CSV schema and data dictionary	Didactic semantics
Reviewability	HTML indexes and browser demos	Safe replay

labeled didactic spoof packets into that external trace for visualization and workflow validation.

Experiment 006 uses a 600.06-second virtual run with 180 seconds baseline, 120 seconds spoofing, 120 seconds flooding, and 180 seconds recovery. Matrix 007 uses the same phase structure across 10 runs. It should be interpreted as deterministic fast-run event simulation: timestamps follow the declared schedule, but the generator does not wait for real wall-clock execution. The internal environment used Python 3.12.3 and `python-can` 4.6.1. Validation uses `dataset/scripts/validate_hashes.py`. Experiment 010 adds a dependency-light wall-clock timing capture using real sleeps and monotonic timestamps. It produced 1,167 software-only frames over 20.000022 seconds, four arbitration IDs, mean inter-arrival time of 0.017153 seconds, and zero negative inter-arrival values. Experiment 011 uses the installed python-can 4.6.1 virtual bus to generate 1,750 frames over 29.998787 seconds, with mean send-to-receive latency of 0.000104 seconds. Experiment 012 replays all 6,158 normalized ICSim frames through a python-can virtual bus over 3.256474 seconds, with mean send-to-receive latency of 0.000042 seconds. These experiments validate timing preservation and tool-backed virtual replay, not physical CAN behavior.

TABLE IV
EVIDENCE PACKAGE COVERAGE

Exp.	Purpose	Frames	Manifest	Labels	Demo	Main limitation
005	Short internal proof-of-concept	2,088	Yes	Yes	Yes	Short; synthetic
006	10-minute internal run	40,200	Yes	Yes	Figures	Single seed
007	10-run intensity matrix	435,450	Yes	Yes	No	Fast-run schedule
008	External ICSim ingestion	6,158	Yes	No attack labels	Yes	Compatibility only
009	ICSim + spoof injection	6,203	Yes	Injected labels	Yes	Didactic injection
010	Wall-clock software capture	1,167	Yes	Baseline	No	Not physical CAN
011	python-can virtual bus	1,750	Yes	Baseline	No	Virtual bus
012	python-can ICSim replay	6,158	Yes	Replay	No	Virtual replay

TABLE V
MATRIX 007 SUMMARY BY INTENSITY

Spoof	Flood levels	Runs	Flood frames	Speed mean
Low	L/M/H	3	6k–24k	155.2
Medium	L/M/H	4	6k–24k	190.0
High	L/M/H	3	6k–24k	225.0

V. RESULTS

A. Internal Labeled Dataset

Across internal labeled experiments, the normalized dataset contains 477,738 frames. Phase counts are 123,977 baseline frames, 78,986 spoofing frames, 151,038 flooding frames, and 123,737 recovery frames. The most frequent IDs are `0x100`, `0x188`, `0x120`, and `0x300`, representing stable simulated traffic. Flooding adds many low-count randomized IDs, increasing arbitration-ID diversity.

The decoded speed on ID `0x100` remains in a didactic normal range during baseline and recovery and increases during spoofing. Across the combined internal dataset, baseline speed ranges from 27 to 92 km/h with mean 61.04 km/h; recovery has mean 61.56 km/h; spoofing ranges from 145 to 240 km/h with mean 203.72 km/h. These values are not vehicle-realism claims; they provide controlled contrast for teaching and trace review.

B. Matrix 007

Matrix 007 added 435,450 frames over 6,000 seconds of synthetic schedule. It varies spoofing intensity and flooding intensity across low, medium, and high profiles. Flooding profiles produce approximately 6,000, 12,000, or 24,000 flooding frames per 120-second flooding phase. Mean spoofed speed is approximately 155 km/h in low profiles, 190 km/h in medium profiles, and 225 km/h in high profiles.

C. External ICSim Compatibility

Experiment 008 imports the public ICSim `data/sample-can.log` trace from commit `2b3333ef866987d8adc9c15ff15b9b9189edf85b`. The upstream SHA-256 is preserved in the source-metadata file and manifest. The importer preserves the raw log, source metadata, license information, normalized CSV, summary, and manifest.

The imported trace contains 6,158 frames over 3.257991 seconds, 34 unique arbitration IDs, and payload byte entropy of 3.361537 bits. Mean inter-arrival time is 0.000529152 seconds, with no negative values. Since the upstream sample does not include attack labels in this package, this experiment is used for external-source compatibility and replay-readiness, not IDS evaluation.

D. ICSim Spoof Injection Demo

Experiment 009 injects 45 labeled didactic packets into the external ICSim trace: 33 speed packets on ID `0x244`, seven door packets on ID `0x19B`, and five turn-signal packets on ID `0x188`. The resulting package has 6,203 frames. The visual dashboard highlights injected frames, displays speed, door state, turn signals, a live CAN frame table, and frame density. This demonstrates safe review of injected events without claiming real-vehicle exploitation.

E. Wall-clock and python-can Virtual-Bus Evidence

Experiment 010 addresses a narrow reproducibility concern left by deterministic fast-run experiments. It generated 1,167 software-only CAN-like frames over 20.000022 seconds using wall-clock sleeps and monotonic timestamps, with zero negative inter-arrival values. Experiment 011 then uses the installed python-can virtual bus to generate and receive 1,750 CAN messages over 29.998787 seconds. Its mean inter-arrival time is 0.017152 seconds and its mean send-to-receive latency is 0.000104 seconds. Experiment 012 replays the external ICSim trace through the same python-can virtual-bus mechanism: all 6,158 input frames are replayed, 34 arbitration IDs are preserved, mean inter-arrival is 0.000529 seconds, and mean send-to-receive latency is 0.000042 seconds. These results do not validate physical CAN arbitration or electrical behavior; they demonstrate that the evidence workflow can preserve real elapsed timing and virtual-bus replay semantics.

VI. EVALUATION AGAINST RESEARCH QUESTIONS

Table VI maps each research question to evidence and limits. This form is intentionally conservative: a claim is included only when an artifact package supports it.

VII. TRIAGE SANITY CHECK

A rule-based heuristic is retained only as a forensic triage sanity check. The rules classify flooding using high arbitration-ID cardinality or unknown-ID ratio, spoofing using decoded

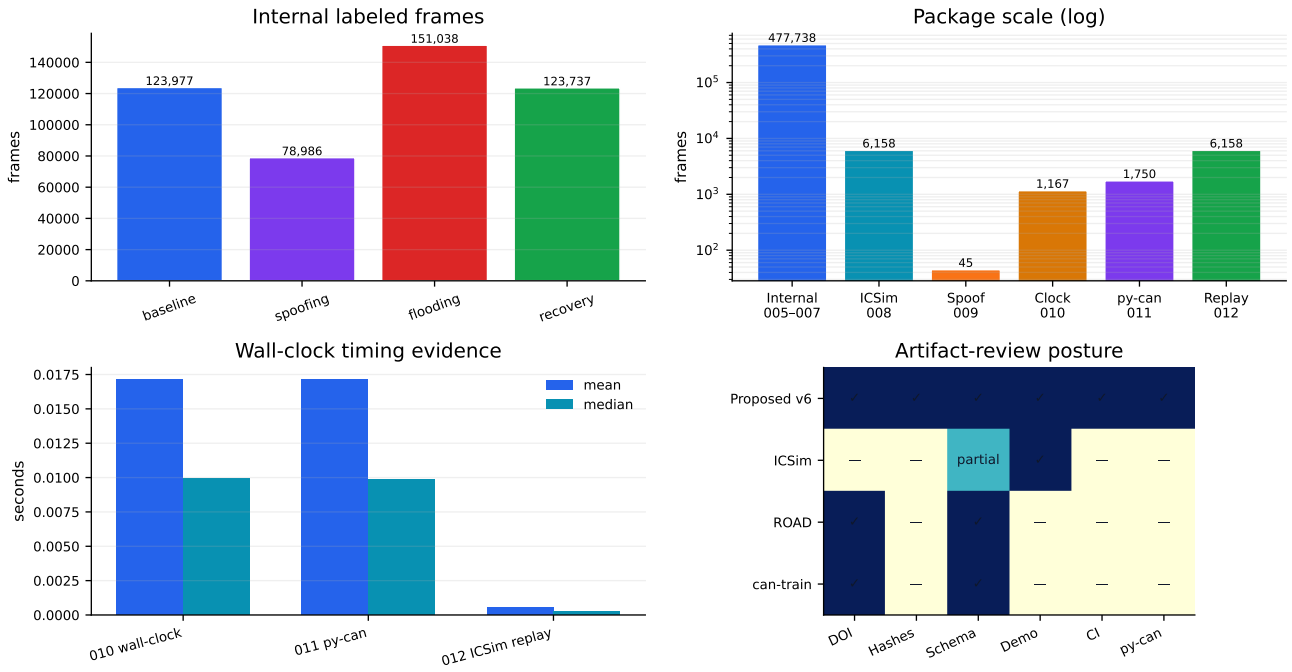

Fig. 2. Evidence coverage dashboard for the v5 artifact. The figure summarizes internal phase balance, external and injected packages, didactic speed contrast, and artifact-review posture.

TABLE VI
RESEARCH QUESTION EVALUATION MATRIX

RQ	Claim	Evidence	Limitation
RQ1	Evidence can be preserved for later review.	Raw/source files, normalized CSV, summaries, SHA-256 manifests, DOI-backed repository.	File-level integrity, not full forensic standard certification.
RQ2	Controlled spoofing/flooding variation is supported.	Exp. 006/007 plus Exp. 010/011/012; 477,738 internal frames; 10-run matrix; wall-clock and python-can virtual-bus captures.	Synthetic labels and fast-run timing.
RQ3	External traces can enter the same pipeline.	Exp. 008 imports 6,158 ICSim frames with source metadata/hash/license.	Compatibility evidence only; not vehicle validation.
RQ4	Review can occur without live hardware.	Observer-safe demos and HTML indexes; Exp. 009 injected-event replay.	Demos are didactic, not live CAN exercises.

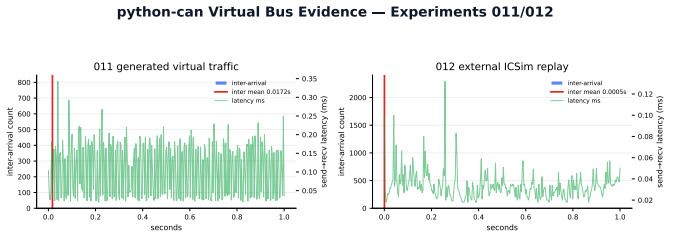

Fig. 3. python-can virtual-bus timing evidence from Experiments 011 and 012: generated traffic and external ICSim replay with send-to-receive latency profiles.

speed above a didactic threshold on ID `0x100`, and other windows as normal. Because the rules are intentionally aligned with the didactic generation process, high performance indicates label and workflow consistency rather than real-world IDS robustness.

Across the combined internal labeled dataset, 6,615 one-second windows were evaluated. The heuristic achieved 0.9998 accuracy and 0.9998 macro-F1, with one normal window flagged as flooding. These values are secondary artifact-validation evidence and should not be interpreted as an intrusion-detection benchmark.

VIII. ARTIFACT EVALUATION CHECKLIST

The public artifact is intended to satisfy common artifact-review expectations. It is available through GitHub and Zenodo, functional through documented scripts and browser demos, reusable through normalized CSV and metadata files,

TABLE VII
RULE-BASED TRIAGE SANITY-CHECK MATRIX

Expected	Normal	Spoofing	Flooding
Normal	3968	0	1
Spoofing	0	1323	0
Flooding	0	0	1323

TABLE VIII
ARTIFACT EVALUATION CHECKLIST

Criterion	Evidence
Available	GitHub repository and Zenodo DOI
Functional	Validation script and browser demos
Reusable	CSV schema, summaries, source metadata
Documented	README, ARTIFACT, ETHICS, SECURITY
Integrity-aware	SHA-256 manifests and source hashes
CI-ready	GitHub Actions validate hashes and build LaTeX
No special hardware	Software-only experiments and replay

and auditable through SHA-256 manifests. It does not require special automotive hardware for the reported software-defined experiments. Table VIII summarizes the checklist.

IX. DISCUSSION

The artifact is strongest as a reproducible forensic-readiness package. A reviewer can inspect plans, raw/source logs, normalized CSV files, source metadata, manifests, summaries, and safe demos. This differentiates the work from a closed simulator or a detector script: the paper's core claim is evidence packaging and reproducible review rather than detection novelty.

Experiment 008 directly addresses the synthetic-only critique by showing that an external ICSim trace can be preserved, normalized, hashed, summarized, and replayed through the same workflow. Experiment 009 extends this into a didactic injection package, demonstrating how labeled events can be added to an external trace for teaching and visualization while preserving provenance and boundaries.

The main trade-off remains fidelity. Physical benches and real datasets provide stronger realism; the proposed artifact provides safety, cost control, transparent semantics, and artifact reviewability. The intended progression is incremental: software-defined reproducibility first, then SocketCAN/`vcan` wall-clock capture, additional public dataset import, and finally HIL validation.

X. THREATS TO VALIDITY

Internal validity. Internal labels are generated by the same framework that creates internal traces. Matrix 007 adds parameter variation, and Experiments 008 and 009 add external-source compatibility and separately labeled injected packets. Nevertheless, internal triage metrics remain label-consistency evidence only.

External validity. Software-only traces do not model physical CAN arbitration, electrical noise, termination, bus-off behavior, gateway filtering, ECU timing constraints, or OEM signal accuracy. The external ICSim import improves compatibility evidence but does not replace real-vehicle or HIL validation.

Construct validity. Forensic readiness is operationalized as traceability, integrity, repeatability, normalization, and reviewability. This is a practical artifact definition rather than a complete forensic standard.

Conclusion validity. High triage scores could be overinterpreted if reported as detection performance. The paper therefore frames the heuristic as a sanity check and keeps it secondary to artifact reproducibility.

Reproducibility. The project provides scripts, manifests, a GitHub repository, and a Zenodo DOI. Reproducibility still depends on readers using the documented artifact release rather than local unpublished files.

XI. LIMITATIONS AND FUTURE WORK

The current artifact is software-defined and observer-safe. It does not claim physical CAN fidelity, production-vehicle attack realism, or operational IDS performance. Future work includes SocketCAN/`vcan` capture with kernel interfaces, ROAD or can-train-and-test import into the same schema, larger multi-run wall-clock timing campaigns, and HIL validation with documented legacy ECUs.

XII. CONCLUSION

This paper presented a reproducible software-defined CAN testbed for automotive cybersecurity and forensic-readiness analysis. The artifact generates labeled internal traces, expands them through a 10-run intensity matrix, imports an external ICSim trace, supports didactic spoof injection, preserves evidence through manifests and hashes, and provides observer-safe browser demos. The v6 framing avoids overclaiming detection or physical realism and emphasizes reproducible, low-cost, evidence-aware learning and review.

ARTIFACT AVAILABILITY

The software, experiment scripts, normalized datasets, manifests, validation scripts, paper drafts, and observer-safe demos are available in the public GitHub repository: https://github.com/ojaneri/can-forensic-readiness-testbed.
The archival Zenodo concept DOI for all artifact versions is https://doi.org/10.5281/zenodo.20541106 (DOI: `10.5281/zenodo.20541106`). The package includes SHA-256 manifests and instructions to inspect Experiments 005–009.

REFERENCES

[1] O. J. Filho, "Can forensic-readiness testbed," Zenodo software artifact, 2026. [Online]. Available: https://doi.org/10.5281/zenodo.20541106

[2] K. Koscher, A. Czeskis, F. Roesner, S. Patel, T. Kohno, S. Checkoway, D. McCoy, B. Kantor, D. Anderson, H. Shacham, and S. Savage, "Experimental security analysis of a modern automobile," in *2010 IEEE Symposium on Security and Privacy*, 2010, pp. 447–462.

[3] S. Checkoway, D. McCoy, B. Kantor, D. Anderson, H. Shacham, S. Savage, K. Koscher, A. Czeskis, F. Roesner, and T. Kohno, "Comprehensive experimental analyses of automotive attack surfaces," in *Proceedings of the 20th USENIX Security Symposium*. USENIX Association, 2011.

[4] C. Miller and C. Valasek, “Remote exploitation of an unaltered passenger vehicle,” Black Hat USA, Tech. Rep., 2015. [Online]. Available: https://illmatics.com/Remote%20Car%20Hacking.pdf
[5] M. Bozdal, M. Samie, S. Aslam, and I. Jennions, “Evaluation of can bus security challenges,” *Sensors*, vol. 20, no. 8, p. 2364, 2020.
[6] S.-F. Lokman, A. T. Othman, and M.-H. Abu-Bakar, “Intrusion detection system for automotive controller area network: A review,” *EURASIP Journal on Wireless Communications and Networking*, vol. 2019, no. 1, p. 184, 2019.
[7] *ISO/SAE 21434: Road vehicles — Cybersecurity engineering*, ISO/SAE Std., 2021.
[8] *UN Regulation No. 155 — Cyber security and cyber security management system*, United Nations Economic Commission for Europe Std., 2021.
[9] OpenGarages, “ICSim: Instrument cluster simulator,” GitHub repository, 2016. [Online]. Available: https://github.com/zombieCraig/ICSim
[10] Caring Caribou Project, “Caring caribou: A friendly car security exploration tool,” GitHub repository. [Online]. Available: https://github.com/CaringCaribou/caringcaribou
[11] C. Smith, “The car hacker’s handbook: A guide for the penetration tester,” 2016.
[12] A. K. T. Yeo, M. E. Garbelini, S. Chattopadhyay, and J. Zhou, “Vitrobench: Manipulating in-vehicle networks and cots ecus on your bench,” *Vehicular Communications*, vol. 43, p. 100649, 2023.
[13] D. Shi, L. Kou, C. Huo, and T. Wu, “A can bus security testbed framework for automotive cyber-physical systems,” *Wireless Communications and Mobile Computing*, vol. 2022, no. 1, p. 7176194, 2022.
[14] M. E. Verma, R. A. Bridges, M. D. Iannacone, S. C. Hollifield, P. Moriano, S. C. Hespeler, B. Kay, and F. L. Combs, “A comprehensive guide to can ids data and introduction of the road dataset,” *PLOS ONE*, vol. 19, no. 1, p. e0296879, 2024.
[15] B. Lampe and W. Meng, “can-train-and-test: A curated can dataset for automotive intrusion detection,” *Computers & Security*, vol. 140, p. 103777, 2024.
[16] can-sleuth Authors, “can-sleuth: Investigating and evaluating automotive intrusion detection datasets,” ACM artifact / project documentation, 2024. [Online]. Available: https://github.com/Trustworthy-AI-Group/can-sleuth